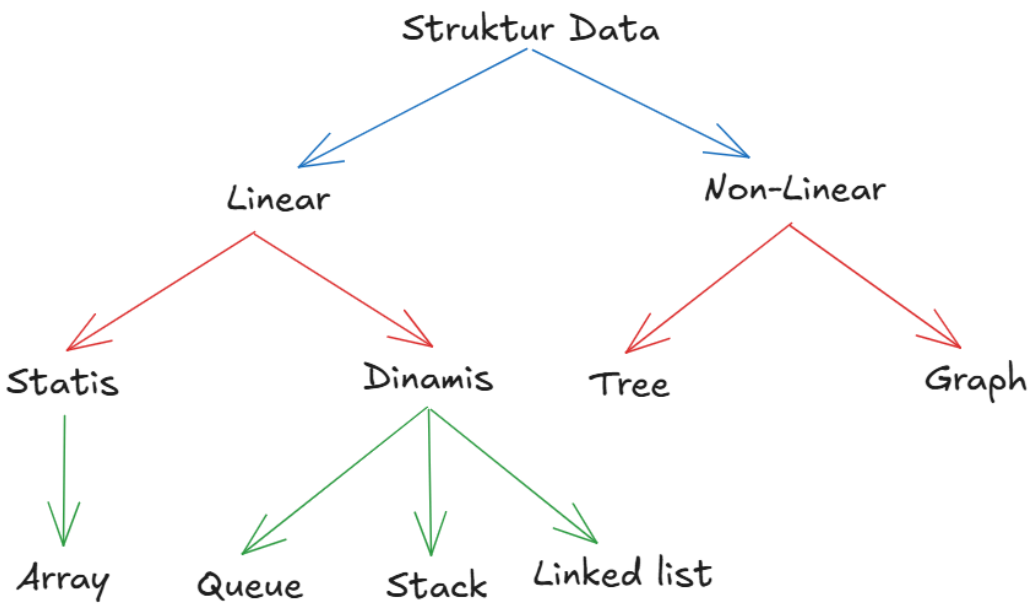
Array & Pointer

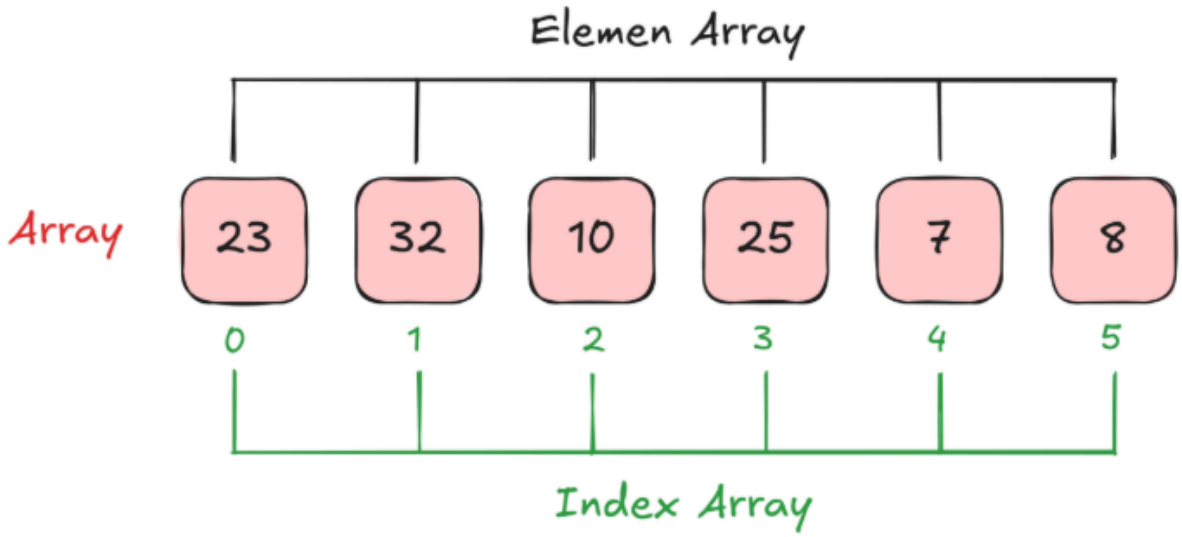
1. Pengantar Singkat Struktur Data

Data merupakan kumpulan dari suatu abjad, simbol yang berbeda untuk merepresentasikan/mewakili sebuah informasi. Struktur data merupakan suatu cara mengorganisasi serta menyimpan data dalam komputer agar dapat diakses dan dikelola secara efisien. Berikut ini beberapa jenis struktur data



2. Array

Array adalah kumpulan elemen dengan tipe data yang sama yang disimpan dalam lokasi memori yang berdekatan. Dengan menggunakan array, kita dapat menyimpan beberapa nilai dalam satu variabel, sehingga memudahkan pengelolaan dan manipulasi data terkait. Setiap elemen dalam array diakses menggunakan indeks, dengan elemen pertama berada pada indeks 0.



Array memiliki karakteristik seperti:

- Ukuran Tetap

  Setelah ukuran array ditetapkan, ukuran tersebut tidak dapat diubah lagi. Ukuran array ditentukan pada saat kode dikompilasi, yang berarti jumlah elemen dalam array harus ditentukan pada saat deklarasi array.

- Bertipe Data Sama

  Semua elemen yang berada pada array harus memiliki tipe yang sama, contohnya, semua bertipe integer, float, atau char.

- Lokasi Memori Dekat

  Elemen array tersimpan pada lokasi memori yang berurutan, sehingga untuk mengakses array sangatlah mudah.

Deklrasi dan inisiasi Array pada C++

Pendeklarasian Array diawali dengan tipe data dari Array tersebut, nama array, dan diikuti ukuran.

```
  // sebuah array berisi 2 integer, angka[0], angka[1],.. angka[9]
   int angka[10];
   // sebuah array berisi 32 pointer, a[0], a[1],.. a[63]
   char* a[64];
   // sebuah cara deklarasi yang salah
   void f(int i) {
   int v1[i];
   }
```

Untuk inisialisasi Array dapat dengan menambahkan "=" setelah deklarasi diikuti dengan "{}" untuk mengisi elemen kedalam Array.

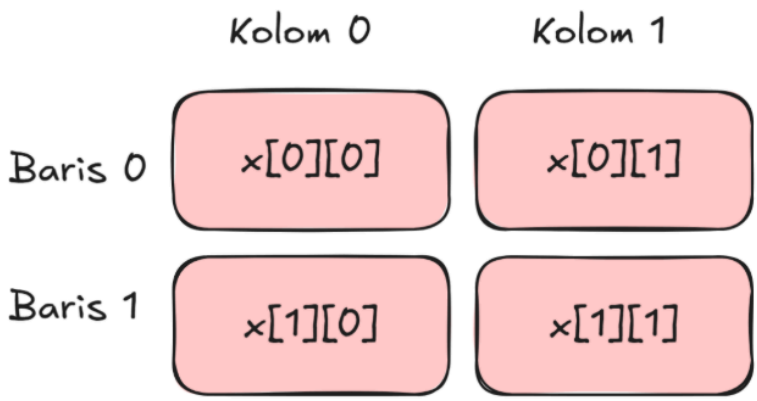
```
// inisialisasi dengan menggunakan nilai
int numbers[8] = {1, 2, 3, 4, 5, 6, 7, 8};

// tanpa menggunakan ukuran
int numbers[] = {1, 2, 3, };

/* inisialisasi array tanpa nilai
lengkap, sehingga ekuivalen
int numbers[4] = {1, 2, 0, 0};  */
int numbers[4] = {1, 2};

// inisialisasi dengan nilai 0
int numbers[5] = {0};
/* inisialisasi yang berisikan terlalu
banyak nilai akan menghasilkan error */

int numbers[10] = {1, 2, 3, 4, 5, 6};
    return 0;
}
```

Array Multidimensi

Array yang dideklarasikan dengan lebih dari satu dimensi disebut array multidimensi. Array multidimensi yang paling banyak digunakan adalah array 2D dan array 3D. Array ini umumnya direpresentasikan dalam bentuk baris dan kolom.

- Array 2 Dimensi

  Array dua dimensi adalah pengelompokan elemen yang disusun dalam baris dan kolom. Setiap elemen diakses menggunakan dua indeks: satu untuk baris dan satu untuk kolom, yang membuatnya mudah divisualisasikan sebagai tabel.




```
#include <iostream>
using namespace std;

int main() {
    int arr[5][5];

    // Mengisi array
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            arr[i][j] = i + j;
        }
    }
    // Menampilkan array
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
```

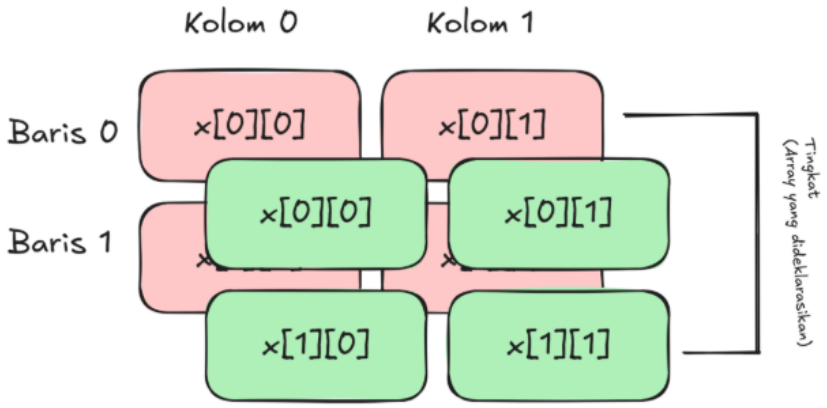
```
    }
    return 0;
}
```

Pada kode di atas, program mendeklarasikan sebuah array dua dimensi berukuran 4 baris dan 4 kolom. Selanjutnya, array tersebut diisi menggunakan perulangan bersarang (nested loop), di mana setiap elemen array diberi nilai hasil penjumlahan indeks baris dan kolomnya, yaitu i + j. Setelah proses pengisian selesai, program mencetak seluruh isi array 2D ke layar dengan menggunakan nested loop yang sama, sehingga nilai array ditampilkan dalam bentuk matriks.

- Array 3 Dimensi

Array tiga dimensi berisikan kumpulan berbagai susunan dua dimensi yang

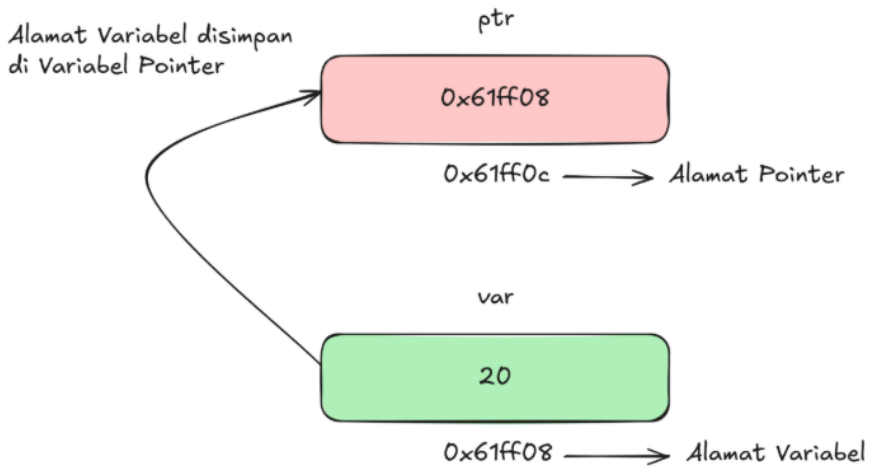
ditumpuk satu di atas yang lain, sehingga dapat dipresentasikannya sebagai Tiga indeks yaitu indeks baris, indeks kolom, dan indeks tingkat yang digunakan untuk mengidentifikasi setiap elemen dalam susunan tiga dimensi secara unik.



```
#include <iostream>
using namespace std;

int main() {
    int arr[3][3][3];

    // Mengisi array 3D dengan pola berbeda
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            for (int k = 0; k < 3; k++) {
                arr[i][j][k] = (i + 1) * (j + k + 1);
            }
        }
    }
    // Menampilkan isi array 3D
    for (int i = 0; i < 3; i++) {
        cout << "Layer ke-" << i << endl;
        for (int j = 0; j < 3; j++) {
            for (int k = 0; k < 3; k++) {
                cout << arr[i][j][k] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

3. Pointer

Pointer adalah sebuah variabel atau objek yang menunjuk ke variabel atau objek lainnya, dalam hal ini pointer bisa dikatakan sebagai sebuah alias dari variabel lainnya. Pointer hanyalah sebuah variabel yang menyimpan alamat memori, memori tersebut berasal dari variabel, objek, dan lainnya.



Dengan menggunakan pointer maka sangat memungkinkan bagi kita untuk menunjuk suatu memori, mendapatkan isi memori dan mengubah isi memori yang ditunjuk.

Operator pada Pointer

Dalam C++, operator yang bekerja dengan pointer sangat penting untuk memanipulasi alamat memori dan nilai yang disimpan di dalamnya. Ada dua jenis operator utama yang sering digunakan dalam konteks pointer, yaitu Address-Of Operator dan Dereference Operator. Berikut adalah penjelasan lebih rinci tentang masing-masing operator:

- Address-Of Operator

Address-of Operator (&) adalah operator yang memungkinkan kita untuk mendapatkan/melihat alamat memori yang dimiliki oleh variabel tersebut. Cara menggunakannya adalah dengan meletakan tanda & di depan identitas saat pemanggilan variabel. Hal itu akan membuat compiler memberikan alamat memori bukan isi/nilai dari memori tersebut. Contoh penggunaan Address-Of Operator sebagai berikut.

```
#include <iostream>
using namespace std;

int main() {
    int angka = 50;
    int *pAngka;

    // Pointer menyimpan alamat variabel angka
    pAngka = &angka;

    // Menampilkan nilai dan alamat
    cout << "Nilai variabel angka   : " << angka << endl;
    cout << "Alamat variabel angka : " << &angka << endl;
    cout << "Alamat yang disimpan pointer : " << pAngka << endl;
    cout << "Nilai yang diakses melalui pointer : " << *pAngka << endl;

    return 0;
}
```

Pada kode di atas, program mendeklarasikan sebuah variabel integer dan sebuah pointer. Pointer tersebut menyimpan alamat dari variabel integer menggunakan operator &. Selanjutnya, program menampilkan nilai variabel, alamat memori variabel, alamat yang disimpan oleh pointer, serta nilai variabel yang diakses melalui pointer menggunakan operator dereference (*).

- Deference operator

  Jika pada address-of operator memungkinkan kita untuk melihat alamat dari variabel yang telah kita buat, maka untuk dereference operator ini berlaku

kebalikannya di mana operator ini memungkinkan kita untuk melihat nilai yang tersimpan pada alamat memori. Contohnya adalah sebagai berikut.

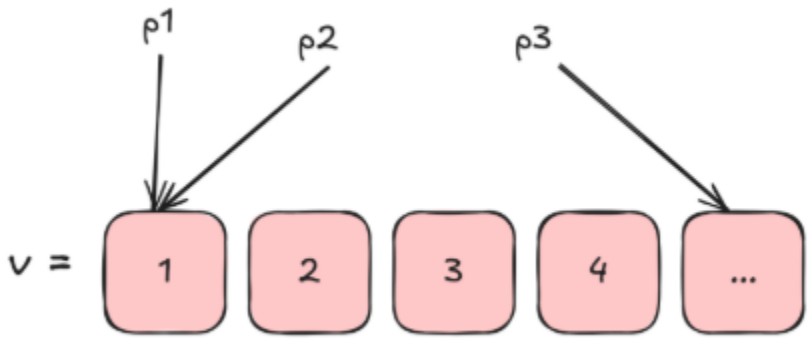
```
#include <iostream>
using namespace std;

int main() {
    int nilai = 10;      // Variabel integer
    int *pNilai;         // Deklarasi pointer

    pNilai = &nilai;     // Pointer menyimpan alamat variabel nilai

    // Menampilkan nilai melalui pointer
    cout << *pNilai << endl;

    // Mengubah nilai variabel menggunakan pointer
    *pNilai = 45;

    // Menampilkan nilai variabel setelah diubah
    cout << nilai << endl;

    return 0;
}
```

Pada kode di atas, program mendeklarasikan sebuah variabel integer dan sebuah pointer yang menunjuk ke alamat variabel tersebut. Nilai variabel pertama kali ditampilkan melalui pointer menggunakan operator dereference (*). Selanjutnya, pointer digunakan untuk mengubah nilai variabel, sehingga perubahan tersebut juga tercermin langsung pada variabel aslinya.

Pointer pada Array

Dalam C++, pointer dan array saling terkait erat. Nama array dapat digunakan sebagai pointer ke elemen awalnya. Misalnya:

```
int v[] = {1, 2, 3, 4};
int* p1 = v; // pointer ke elemen paling awal pada array (implisit)
int* p2 = &v[0]; // pointer ke elemen paling awal pada array
int* p3 = &v[4]; // pointer ke satu nilai setelah elemen akhir
```



Ketika membuat sebuah array yang bertipe integer, maka proses alokasi pada memori kita akan berurutan dengan jarak 4 byte sebanyak panjangnya sebuah array. Jadi ketika ingin mengakses array menggunakan pointer kita cukup menyimpan alamat dari data pertama saja, kemudian ketika ingin mengakses data selanjutnya cukup dengan melakukan operasi increment.

```
#include <iostream>
using namespace std;

int main() {
    int data[5] = {10, 20, 30, 40, 50};
    int *p = data;   // Pointer menunjuk ke elemen pertama array

    // Pointer bergerak selama masih berada dalam batas array
    while (p < data + 5) {
        cout << "Nilai: " << *p << ", Alamat: " << p << endl;
        p++;   // Maju ke elemen berikutnya
    }

    return 0;
}
```

Pada kode di atas, program mendeklarasikan sebuah array dan sebuah pointer yang menunjuk ke elemen pertama array tersebut. Pointer kemudian digunakan untuk mengakses setiap elemen array secara berurutan dengan cara menaikkan alamatnya menggunakan operasi p++. Perulangan berhenti ketika pointer mencapai batas akhir array, sehingga seluruh elemen array dapat ditampilkan dengan aman beserta alamat memorinya.

Studi kasus

Seorang dosen ingin membuat sebuah program sederhana untuk membantu mengolah data nilai mahasiswa. Terdapat 5 orang mahasiswa dan nilai mereka sudah disimpan dalam sebuah array bertipe integer. Untuk melatih pemahaman tentang pointer, dosen tersebut meminta agar pengaksesan data nilai tidak menggunakan indeks array, melainkan menggunakan pointer. Instruksinya:

- Buatlah program C++ yang menyimpan nilai 5 mahasiswa dalam sebuah array.
- Gunakan pointer untuk mengakses setiap elemen array tersebut.
- Tampilkan total dan rata-rata nilai mahasiswa.